

Algorithms

Lecture 13:

Graph Traversal

Dr. Wang, Jingwen

Department of Computer Science

Lecture outline

Announcements/notes

- Daily Problems 10-2
- Reading ADM chapter 7.8 – 7.9

Today's lecture

- Review: Graph Data Structures
- Graph Traversal
- Breadth-First Search



Daily Coding Problems 12

Review: Graph Data Structures

Review: Graphs

- Graphs are one of the unifying themes of computer science.
- A graph $\mathbf{G} = (\mathbf{V}, \mathbf{E})$ is defined by a set of vertices $\mathbf{V}$, and a set of edges $\mathbf{E}$ consisting of ordered or unordered pairs of vertices from $\mathbf{V}$.

Review: Flavors of Graphs

- **Directed vs. Undirected** Graphs
- **Weighted vs. Unweighted** Graphs
- **Simple vs. Non-simple** Graphs
- **Sparse vs. Dense** Graphs
- **Cyclic vs. Acyclic** Graphs
- **Directed Acyclic Graph (DAG)**
- **Connected Graph**
- **Complete Graph**

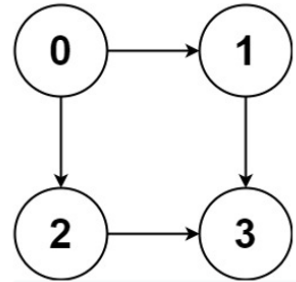
Review

- The **degree** of a vertex of a graph
- The **Clique** subgraph
- A **clique**, C , in an undirected graph $G = (V, E)$ is a subset of the vertices, $C \subseteq V$, such that every two distinct vertices are adjacent.

Review: Data Structures for Graphs

- Edges List

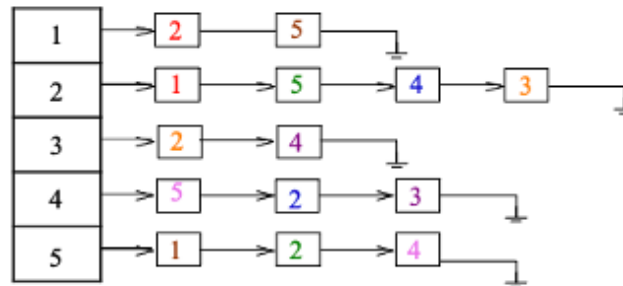
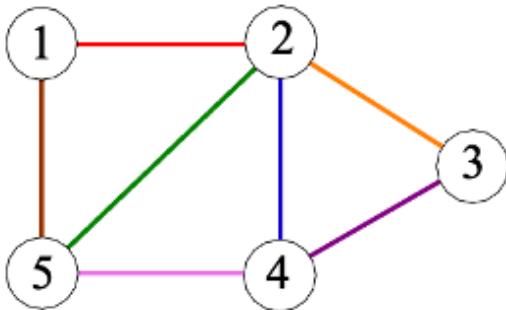
- $E = [[0, 1], [0, 2], [1, 3], [2, 3]]$



- Adjacency matrix

$$M = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

- Adjacency Lists



Tradeoffs Between Adjacency Lists and Adjacency Matrices

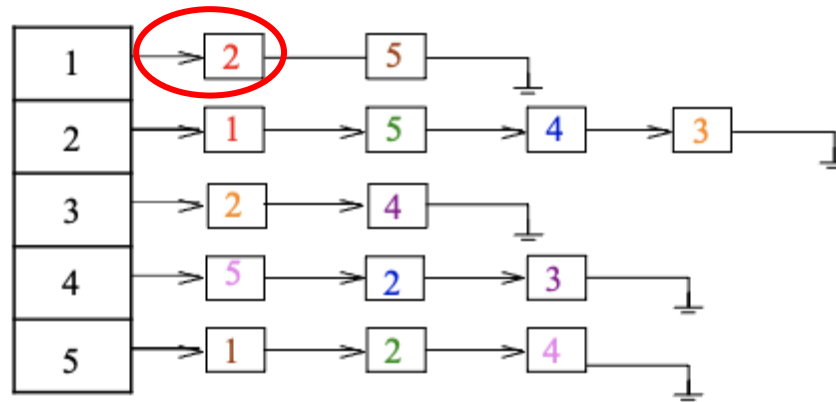
n vertices and m edges

Comparison	Winner
Faster to test if (x, y) exists	matrices
Faster to find vertex degrees	lists
Less memory on sparse graphs	lists $O(m + n)$ vs. $O(n^2)$
Less memory on dense graphs	matrices (small win)
Edge insertion	matrices $O(1)$
Edge deletion	matrices $O(1)$ vs. $O(d_i)$
Faster to traverse the graph	lists $O(m + n)$ vs. $O(n^2)$
Better for most of the problems	lists

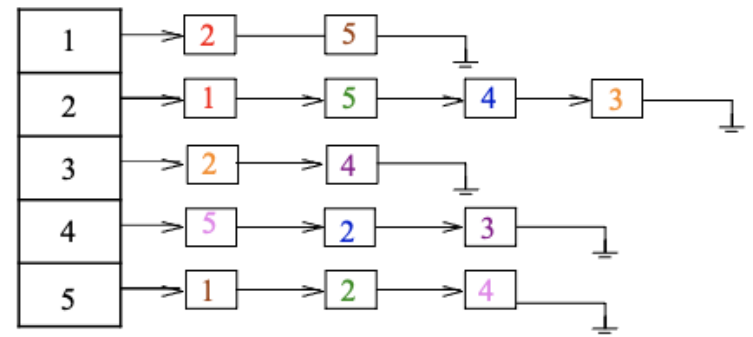
- Both representations are very useful and have different properties, although adjacency lists are probably better for most problems.

Edgenodes in Adjacency Lists

```
typedef struct edgenode {  
    int y;                /* adjacency info */  
    int weight;           /* edge weight, if any */  
    struct edgenode *next; /* next edge in list */  
} edgenode;
```



Graph Representation

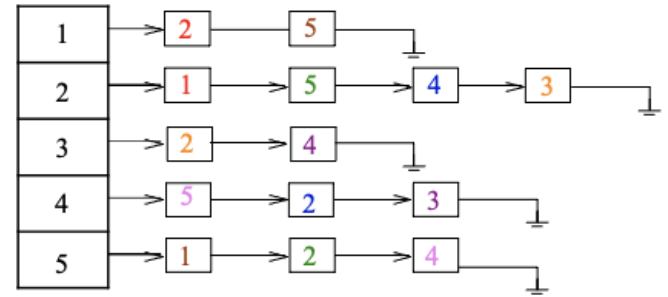


- The vanilla adjacency lists representation:

```
typedef struct {  
    edgenode *edges[n+1]; /* adjacency info */  
    int m;                /* number of edges in the graph */  
} graph;
```

- An undirected edge (x, y) appears twice in any adjacency-based graph structure, once as y in x 's list, and once as x in y 's list.

Graph Representation



- The advanced adjacency lists representation:

```
typedef struct {  
    edgenode *edges[n]; /* adjacency info */  
    int degree[n];      /* outdegree of each vertex */  
    int n;               /* number of vertices in the graph */  
    int m;               /* number of edges in the graph */  
    int directed;        /* is the graph directed? */  
} graph;
```

- The degree field counts the number of meaningful entries for the given vertex.

Problem of the Day

Problem of the Day

- Given an **undirected** graph $G = (V, E)$ with n vertices and m edges, design correct and efficient algorithms to convert between the following graph data structures.
- You must give the time complexity of each algorithm.
- next page continue...

1. Convert from an adjacency matrix to adjacency list.
2. Convert from an adjacency list to an adjacency matrix.

Graph Traversal

Traversing a Graph

- One of the most fundamental graph problems is to traverse every edge and vertex in a graph.
- For *efficiency*, we must make sure we visit each edge at most twice.
- For *correctness*, we must do the traversal in a systematic way so that we don't miss anything.

Recall: All Paths From Source to Target

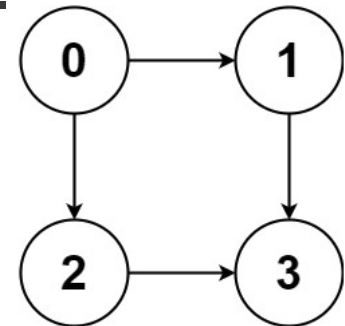
- Given a directed acyclic graph (**DAG**) of n nodes labeled from 0 to $n - 1$, find all possible paths from node 0 to node $n - 1$ and return them in **any order**.
- The graph is given as follows: `graph[i]` is a list of all nodes you can visit from node i (i.e., there is a directed edge from node i to node `graph[i][j]`).

- **Example:**

- **Input:** `graph = [[1, 2], [3], [3], []]`

- **Output:** `[[0, 1, 3], [0, 2, 3]]`

- **Explanation:** There are two paths: $0 \rightarrow 1 \rightarrow 3$ and $0 \rightarrow 2 \rightarrow 3$

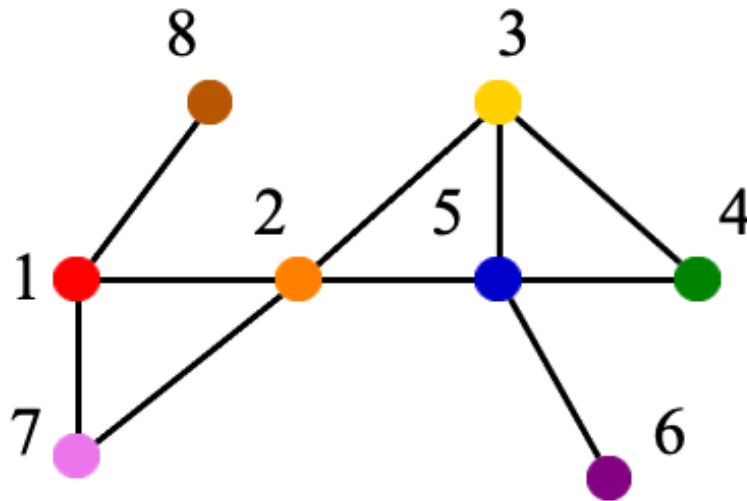


Breadth-First Search (BFS)

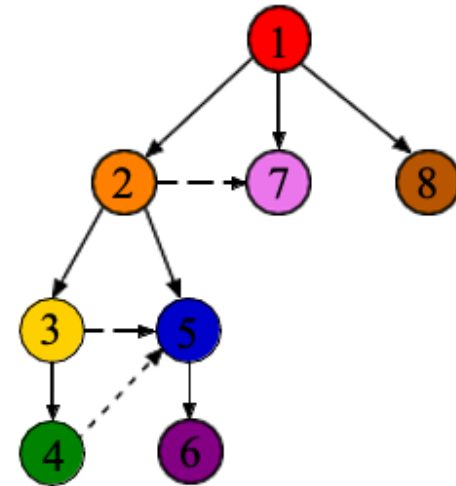
Breadth-First Traversal

- Breadth First Search (BSF)
- The basic operation in most graph algorithms is completely and systematically traversing the graph.
- We want to visit every vertex and every edge exactly once in some well-defined order.

Breadth-First Traversal



A Graph with 8 nodes

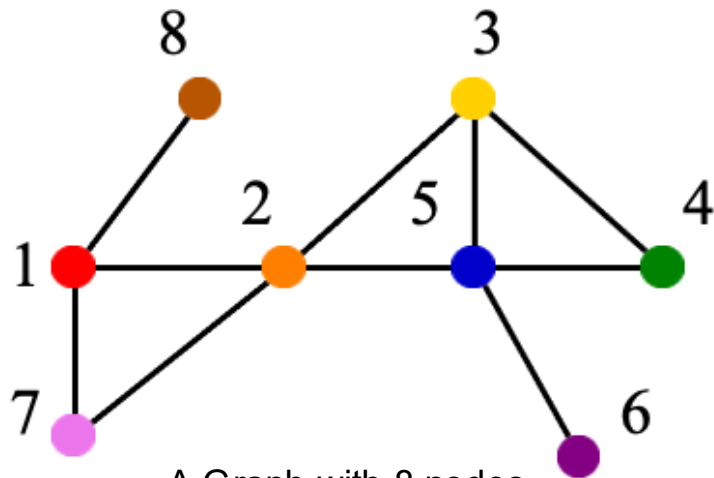


Nodes visited using BFS

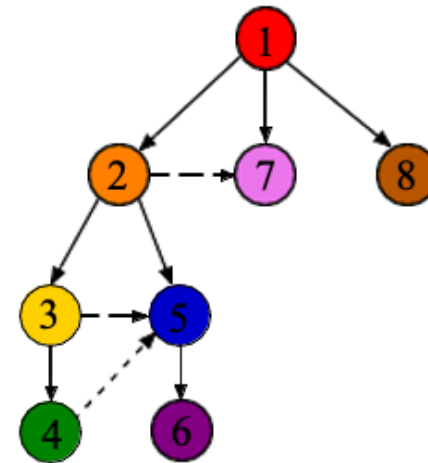
--	--	--	--	--	--	--	--

Vertices States

- Obviously, a vertex cannot be processed before we discover it, so the state of each vertex progresses from *undiscovered* to *discovered* to *processed*.



A Graph with 8 nodes

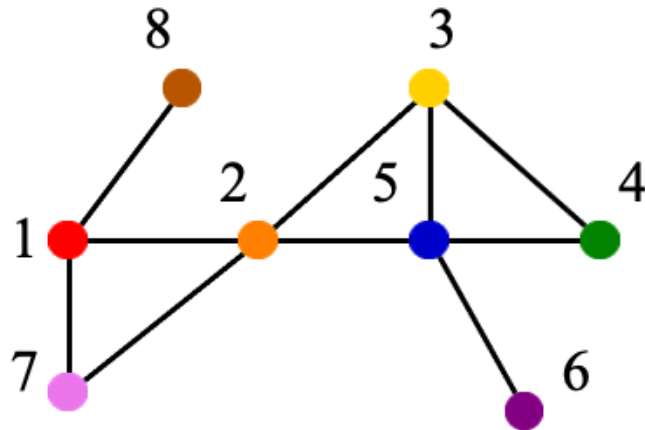


Nodes visited using BFS

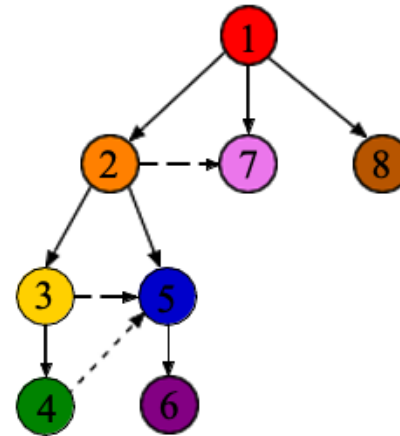
Vertices States

- Each vertex will always be in one of the following three states:
 - ***undiscovered*** – the vertex in its initial, virgin state.
 - ***discovered*** – the vertex after we have encountered it, but before we have checked out all its incident edges.
 - ***processed*** – the vertex after we have visited all its incident edges.

Vertices States Example



A Graph with 8 nodes



Nodes visited using BFS

	1	2	3	4	5	6	7	8
processed								
discovered								

Data Structures for BFS

- We use two Boolean arrays to maintain our knowledge about each vertex in the graph.
- A vertex is `discovered` the first time we visit it.
- A vertex is considered `processed` after we have traversed all outgoing edges from it.

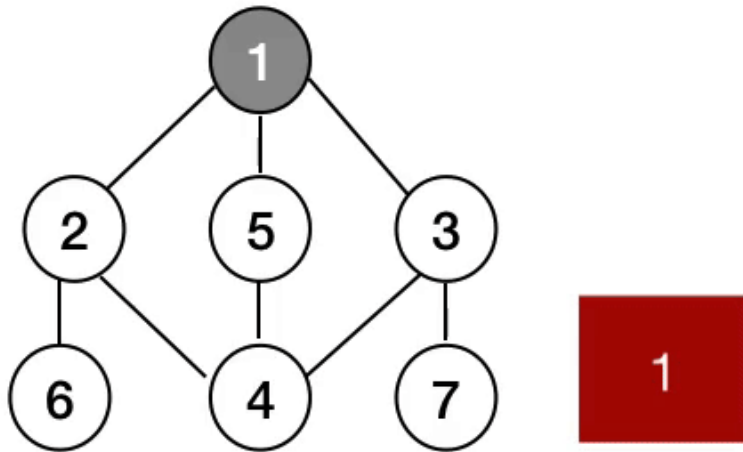
```
bool processed[n+1];  /* which vertices have been processed */  
bool discovered[n+1]; /* which vertices have been found */
```

Initializing BFS

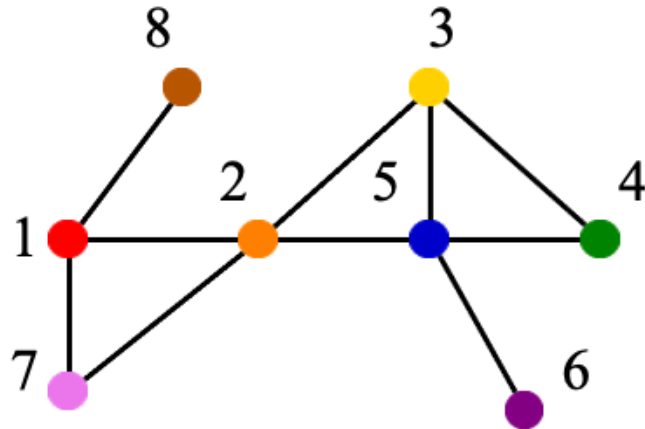
```
void initialize_search(graph *g) {  
    for (int i = 0; i <= g->n; i++) {  
        processed[i] = false;  
        discovered[i] = false;  
    }  
}
```

BFS using Queue

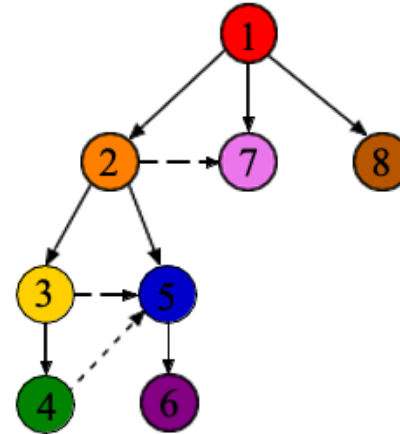
- Once a vertex is discovered, it is placed on a Queue. Thus the oldest vertices / closest to the root are expanded first.



Vertices States Example



A Graph with 8 nodes



Nodes visited using BFS

	1	2	3	4	5	6	7	8
processed								
discovered								

Queue									
-------	--	--	--	--	--	--	--	--	--

```

void bfs(graph *g, int start) {
    queue q; /* queue of vertices to visit */
    int v;    /* current vertex */
    int y;    /* successor vertex */
    edgenode *p; /* temporary pointer */

    init_queue(&q);
    enqueue(&q, start);
    discovered[start] = true;

    while (!empty_queue(&q)) {
        v = dequeue(&q);
        p = g->edges[v];
        while (p != NULL) {
            y = p->y;
            if (!discovered[y]) {
                enqueue(&q, y);
                discovered[y] = true;
            }
            p = p->next;
        }
    }
}

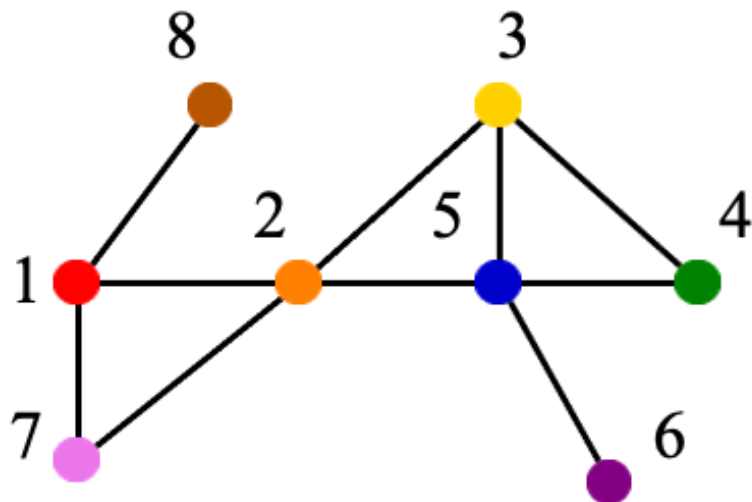
```

BFS running time

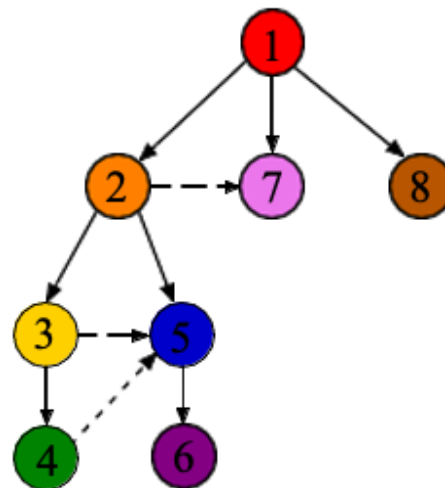
- If we represent the graph G by adjacency matrix, the running time of BFS algorithm is $O(n^2)$, where n is the number of nodes.
- If we represent the graph G by adjacency list, the running time of BFS algorithm is $O(m + n)$, where m is the number of edges and n is the number of nodes.

The Shortest Path Problem

- **Breadth-first search** is appropriate if we are interested in **shortest paths on unweighted graphs**.

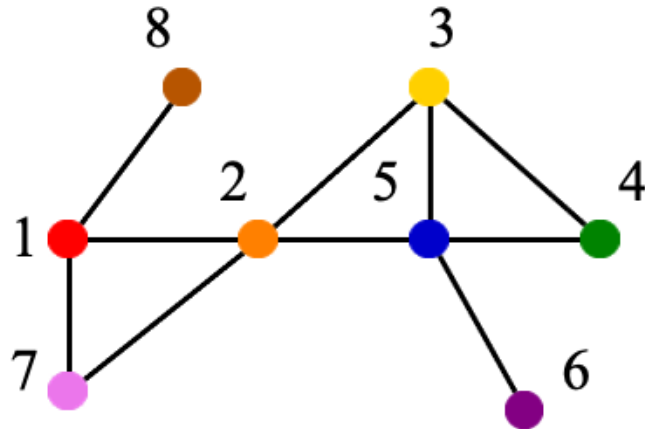


A Graph with 8 nodes

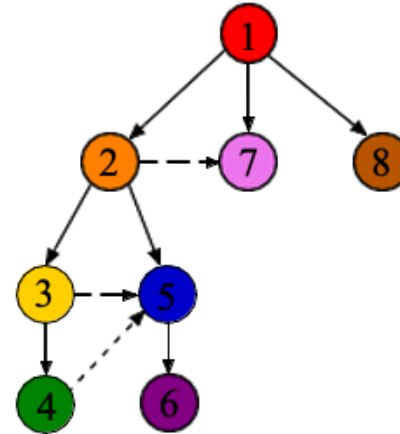


Nodes visited using BFS

Vertices States – parent node



A Graph with 8 nodes



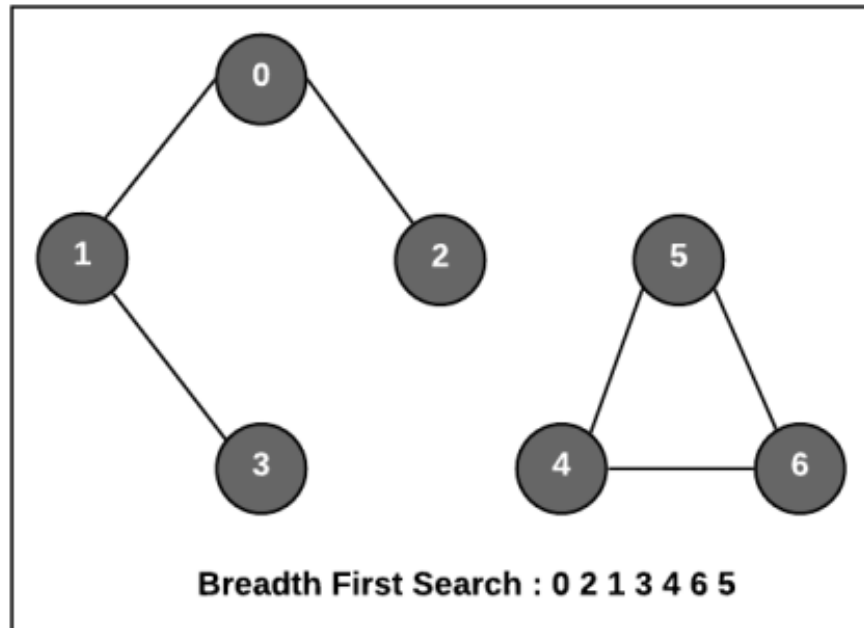
Nodes visited using BFS

	1	2	3	4	5	6	7	8
processed								
discovered								
parent								

An Application of BFS

BFS in Disconnected Graph

- The *connected components* of an undirected graph are the separate “pieces” of the graph such that there is no connection between the pieces.
- Many seemingly complicated problems reduce to finding or counting connected components.



BFS in Disconnected Graph

- Anything we discover during a BFS must be part of the same connected component.
- We then repeat the search from any undiscovered vertex (if one exists) to define the next component, until all vertices have been found.

Implementation

```
void connected_components(graph *g) {  
    int c;                /* component number */  
    int i;                /* counter */  
  
    initialize_search(g);  
  
    c = 0;  
    for (i = 1; i <= g->nvertices; i++) {  
        if (!discovered[i]) {  
            c = c + 1;  
            printf("Component %d:", c);  
            bfs(g, i);  
            printf("\n");  
        }  
    }  
}
```

- running time?



Exercise - 841. Keys and Rooms



▷ *Questions?*